Guide: Why a Fixed Odd-Step Budget Leaves a Gap

Collatz proof project

September 5, 2026

The proposed proof strategy

The previous result reduces the escape problem to seeds whose multiplicative coefficient never drops below one. One possible strategy is to assume a smallest such seed exists, then find a smaller predecessor with the same property.

Lean now proves a limitation of this strategy when the predecessor search uses a fixed maximum number of odd steps. The limitation allows arbitrarily many even steps; it is not just a short-search problem.

What the theorem says

Fix an odd-step budget K. If a target has remainder one on division by 3^K, every positive-length segment reaching it with at most K odd steps has a contracting final coefficient.

Therefore no predecessor whose coefficients never contract can reach that target within the budget. For example, a one-odd-step budget cannot supply such a predecessor for any target congruent to one modulo three. More odd steps can change the situation: $3 \to 5 \to 8 \to 4$ has two odd steps and coefficient $9/8 > 1$.

Why it is true

Reduce a starting seed modulo 2^t, preserving the segment's parity word. Lean proves that the endpoint of this canonical residue is below 3^j, where j is its odd-step count. The original and canonical endpoints have the same remainder modulo 3^j.

If that remainder were one, the canonical endpoint would have to be one. A noncontracting coefficient would then force the canonical seed to be one as well. But a positive return from one to itself has a contracting coefficient. This contradiction gives the obstruction.

What remains possible

The obstructed targets can be arbitrarily large and can avoid multiples of three. Thus adding a finite list of small exceptions cannot repair a fixed-budget coverage claim.

None of those targets is proved to escape or to have coefficients that never contract. The theorem identifies a gap in this particular backward method. A budget that grows with the target, or another argument excluding the residue class, remains possible. The noncontracting-seed exclusion and nontrivial-cycle exclusion required for Collatz are still open.

Verification

The Lean build and 144-declaration standard-foundations audit pass. Run `lake build Collatz.InverseBarrier` from `lean/`.

The technical paper contains the exact statements and proof. No mathematical priority claim is made.